

07 | Github Action：打通IaC与GitOps的桥梁

潘野 · 云原生基础架构实战课



你好，我是潘野。

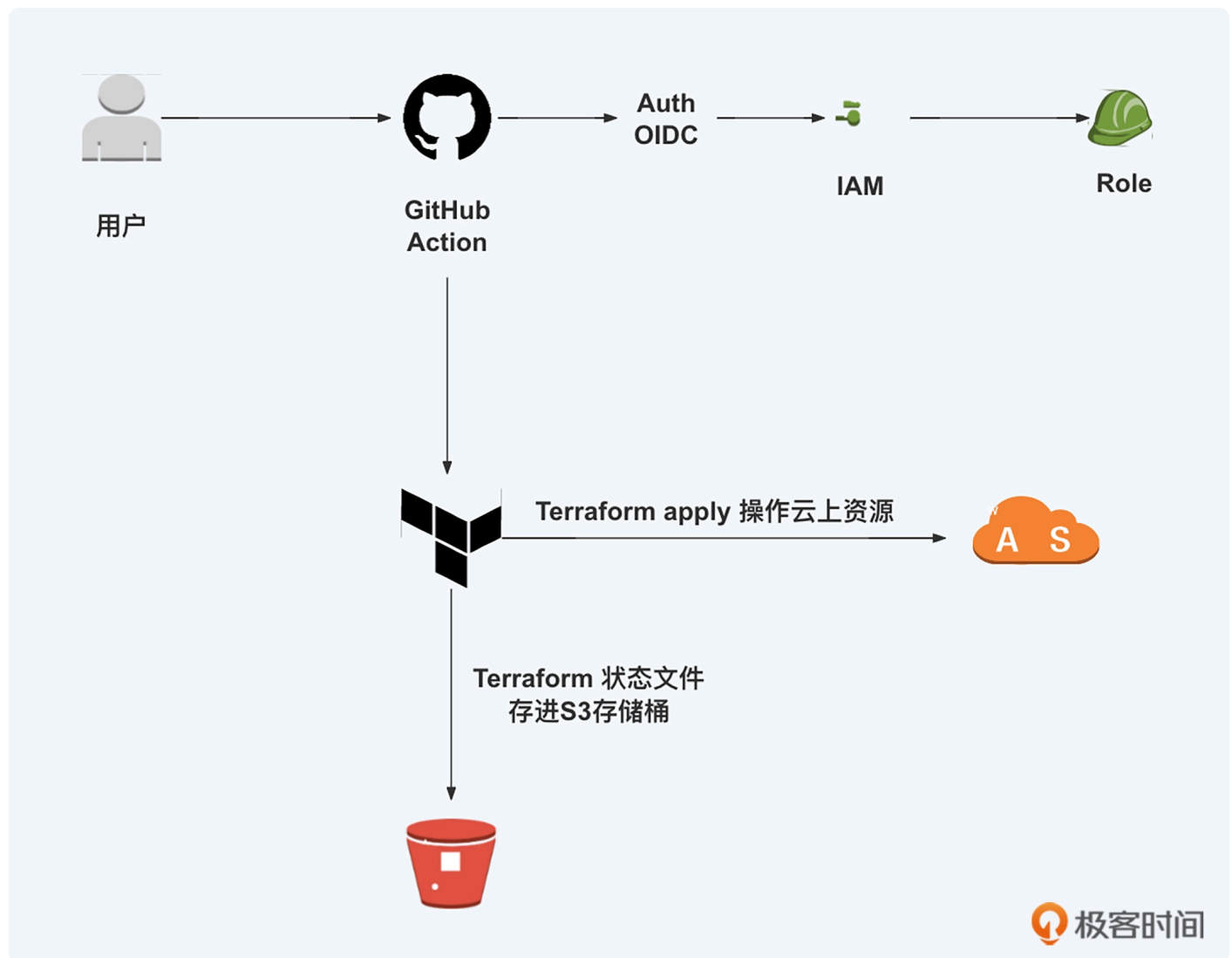
前面的课程里，我们学习了通过持续集成与部署的方式管理 IaC 代码的思路。不过知识必须学以致用，才能真正掌握。接下来的三讲内容，我们就来实操演练，学会如何使用 GitOps，从而更好地管理 IaC 代码。

今天，我们会使用 GitHub Actions 这个工具来管理 Terraform 代码。为什么要选择 GitHub Actions 呢？

之前我们已经了解到，GitOps 方式离不开在 GitHub 里管理代码。GitHub 是一个广泛使用的代码托管平台，提供了许多与代码管理和协作相关的功能。而 GitHub Actions 是 Github 中自带的 CI/CD 工具，上手门槛较低。掌握了这个工具，能让你的管理、部署工作事半功倍。

基于 Github Action 的管理架构

在进入实操演练之前，我们需要先了解一下基于 Github Action 的管理架构长什么样。具体来说，就是通过 Github Action 来管理 Terraform 代码，并自动执行变化的代码，从而更改基础设施的架构。



对照架构图，不难看出 Github Action 的 Terraform 代码的管理主要体现在以下方面：

用户提交 Pull Request 之后，Github Action 通过 OIDC 的方式与 AWS 账户进行认证。通过认证之后，Github Action 执行 `terraform plan` 和 `terraform apply` 两个命令，Terraform 会根据代码去更改基础设施配置。

Terraform 的状态文件会存在对象存储 S3 中，以便追踪 Terraform 的变化。

所以我们要做的是这么几步：

1. 配置 AWS IAM 权限，使得 GitHub Action 可以通过 OIDC 的方式连接上 AWS
2. 创建 S3 存储桶，让 Terraform 通过 Github Action 的流水线将状态文件存储进 S3 中
3. 编写 Github Action 配置，实现使用 Github Action 来执行 Terraform 的操作。

那么接下来，我们就进入实战环节。

GitHub Action 如何与 AWS 做认证？

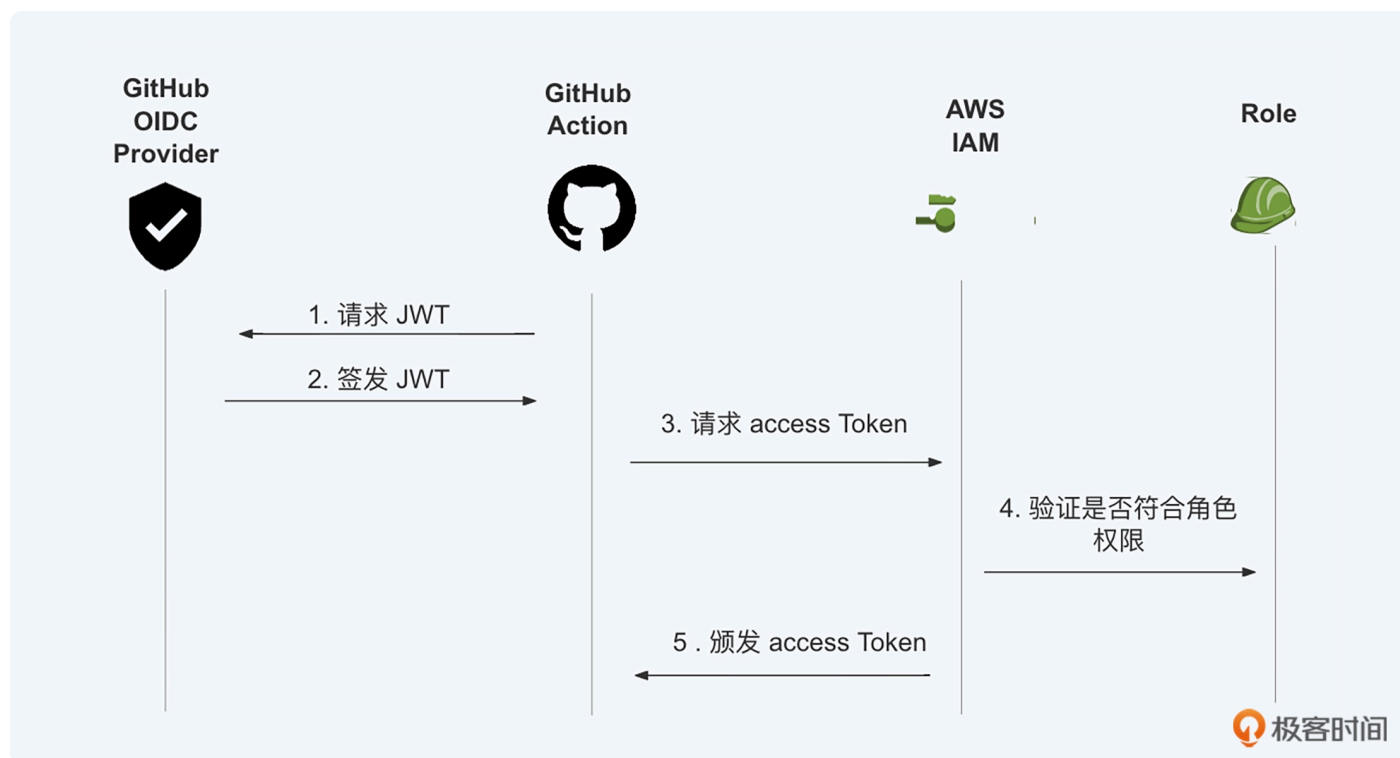
当我们手动执行 `terraform plan` 之前，需要先在 AWS 里的 IAM 中新建一个 USER，并且生成一组 access key，里面包含了 Access Key ID，Secret Access Key。

但是这种方式存在一些弊端。首先是安全隐患。因为 AWS Access Key 是以明文形式生成出来的，交给用户使用。所以，如果在使用的过程中没有规范存放，就有可能会泄露，带来安全风险。

其次，这种方式非常依赖人工检查，比较低效。AWS 建议定期更换 AWS Access Key，并且始终使用最低的必要权限来控制对 AWS Access Key 的访问。那么一旦 Access Key 过期，我们又恰好忘记检查，就会导致自动化程序不可用。

所以，在 GitHub 上，我们可以通过使用 GitHub OIDC (OpenID Connect) 提供程序并仅允许在特定的 GitHub 操作中运行这些凭证。相比将 AWS 的 Access Key 的所有信息存储在 GitHub 密钥中，这样做更加安全且高效。

我们这就来看看 GitHub OIDC 是如何与 AWS 对接做认证的。你可以结合后面的认证流程图听我分析。



图里面序号 1~5 代表认证的操作步骤，我们挨个顺一遍。

1. GitHub Action 向 Github OIDC 提供商请求 JWT（Java Web Token）。
2. GitHub OIDC Provider 将签名的 JWT 发布到我们的 GitHub Action。
3. 带有我们签名的 JWT 的 GitHub Action 将向 AWS 中的 IAM 身份提供商请求临时访问令牌。
4. IAM 身份提供商将会使用 GitHub OIDC 提供商验证签名的 JWT，并验证身份提供商是否可以使用我们想要承担的角色。
5. AWS IAM 基于角色的权限，给 GitHub 颁发临时访问令牌。

经过这些步骤后，我们的 GitHub 操作环境就可以访问 AWS 的相关资源了。

OIDC provider 的整个配置过程也相当简单。如下图所示，我们只需要在 IAM 的身份提供商里填上 GitHub Action 的 token URL 以及受众，然后点击添加提供商即可。

添加身份提供商 信息

配置提供商

提供商类型 信息



SAML

在您的 AWS 账户与符合 SAML 2.0 规范的身份提供商(如 Shibboleth 或 Active Directory Federation Services)之间建立信任。



OpenID Connect

在您的 AWS 账户与身份提供商(如 Google 或 Salesforce)服务之间建立信任。

提供商 URL

为身份验证请求指定安全的 OpenID Connect URL。

`https://token.actions.githubusercontent.com`

获取指纹

最长 255 个字符。URL 必须以“https”开头

受众 信息

指定身份提供商为您的应用颁发的客户端 ID。

`sts.amazonaws.com`

最长 255 个字符。请使用字母数字或“.-/”字符。

添加标签 - 可选 信息

标签是您可以添加到 AWS 资源的键值对，以帮助识别、整理或搜索资源。

没有与资源关联的标签。

添加新标签

最多可以再添加 50 个标签。

取消

添加提供商



最后，我们就会看到生成了一个专门用于 GitHub Action 的身份提供商。这样认证工作就完成了。



如何让 Terraform 将状态文件存到 S3 中？

我们回想一下第 2 讲学过的 Terraform 的状态管理，我曾提到状态文件管理非常重要。所以在 GitHub Action 中，我们需要将 Terraform 的状态文件存在 AWS 的 S3 中。这样既安全地存储我们的管理文件，又能让我们拥有版本追溯的能力。

创建 S3

我们需要先创建一个 S3 存储桶。这里有两种创建方式，我们分别看一下。

第一种方式是登陆到 Web Console 上创建。我们在 AWS 的 Web Console 上进入 S3 界面，点击创建存储桶，然后输入存储桶名称“cloudnative-tfstate-1”。

请注意，在对象所有权这里我们要选择 ACL 已禁用，因为 Terraform 状态的文件也需要安全保护，不能被其他人随意访问到。

创建存储桶 信息

存储桶是在 S3 中存储数据的容器。

常规配置

AWS 区域

亚太地区(香港) ap-east-1

存储桶名称 信息

cloudnative-tfstate-1

存储桶名称在全局命名空间中必须是唯一的，并遵循存储桶命名规则。[参阅存储桶命名规则](#)

从现有存储桶复制设置 – 可选

仅复制以下配置中的存储桶设置。

选择存储桶

格式: s3://bucket/prefix

对象所有权 信息

控制从其他 AWS 账户写入到此存储桶的对象所有权以及访问控制列表 (ACL) 的使用。对象所有权决定谁可以指定对象的访问权限。

☒ ACL 已禁用(推荐)

此存储桶中的所有对象均归此账户所有。仅可使用策略指定对此存储桶及其对象的访问权限。

☐ ACL 已启用

此存储桶中的对象可归其他 AWS 账户所有。可以使用 ACL 指定对此存储桶及其对象的访问权限。

对象所有权

强制存储桶所有者

此存储桶的“屏蔽公共访问权限”设置

通过访问控制列表(ACL)、存储桶策略、接入点策略或三者结合的方式授予对存储桶和对象的公开访问权限。为了确保阻止对此存储桶及其对象的公开访问，请启用“阻止所有公开访问”。这些设置将仅适用于此存储桶及其接入点。AWS 建议您打开“阻止所有公开访问”，但在应用任何这些设置前，请确保您的应用程序在没有公开访问权限的情况下也能正常工作。如果您需要对此存储桶或其中的对象进行某些级别的公开访问，您可以根据自己的特定存储使用案例需求自定义下面的各项设置。[了解更多](#)

☒ 阻止所有公开访问

打开此设置与打开下面全部四个设置相同。下面的每一个设置都彼此独立。

☒ 阻止通过新访问控制列表(ACL)授予的对存储桶和对象的公开访问

S3 将阻止对新增加的存储桶或对象应用的公有访问权限，并且阻止为现有存储桶和对象创建新的公有访问 ACL。此设置不会更改任何允许使用 ACL 公有访问 S3 资源的现有权限。

☒ 阻止通过任何访问控制列表(ACL)授予的对存储桶和对象的公开访问

S3 将忽略向存储桶和对象授予公有访问权限的所有 ACL。

☒ 阻止通过新公有存储桶策略或接入点策略授予的存储桶和对象公有访问

S3 将阻止授权对存储桶和对象进行公有访问的新存储桶策略和接入点策略。此设置不会更改任何允许对 S3 资源进行公有访问的现有策略。

☒ 阻止通过任何公有存储桶策略或接入点策略对存储桶和对象的公有和跨账户访问

S3 将忽略利用授权对存储桶和对象进行公有访问的策略对存储桶或接入点进行的公有和跨账户访问。

存储桶版本控制

版本控制是将某一对象的多个变体保留在同一存储桶中的一种方法。您可以使用版本控制来保留、检索以及还原存储在您的 Amazon S3 存储桶中每个对象的每个版本。通过版本控制，您可以轻松地从意外的用户操作和应用程序故障中恢复。[了解更多](#)

存储桶版本控制

- ☐ 禁用
- ☒ 启用

接着，点击启用存储桶版本，再点击默认加密启用即可。

存储桶版本控制

版本控制是将某一对象的多个变体保留在同一存储桶中的一种方法。您可以使用版本控制来保留、检索以及还原存储在您的 Amazon S3 存储桶中每个对象的每个版本。通过版本控制，您可以轻松地从意外的用户操作和应用程序故障中恢复。[了解更多](#)

存储桶版本控制

- ☐ 禁用
- ☒ 启用

标签 - 可选 (0)

您可以使用桶标签来跟踪存储成本和整理桶。[了解更多](#)

此存储桶没有关联的标签。

添加标签

默认加密 信息

服务器端加密会自动应用于存储在此存储桶中的新对象。

加密类型 信息

- ☒ 使用 Amazon S3 托管密钥进行服务器端加密 (SSE-S3)
- ☐ 使用 AWS Key Management Service (SSE-KMS) 进行服务器端加密
- ☐ 使用 AWS Key Management Service 密钥 (DSSE-KMS) 进行双层服务器端加密
- 使用两层独立的加密层保护您的对象。有关定价的详细信息，请参阅 [Amazon S3 定价](#) 页面的存储选项卡上的 DSSE-KMS 定价。

存储桶密钥

使用 SSE-KMS 的 S3 存储桶密钥可减少对 AWS KMS 的调用，从而降低加密成本。DSSE-KMS 不支持 S3 存储桶密钥。[了解更多](#)

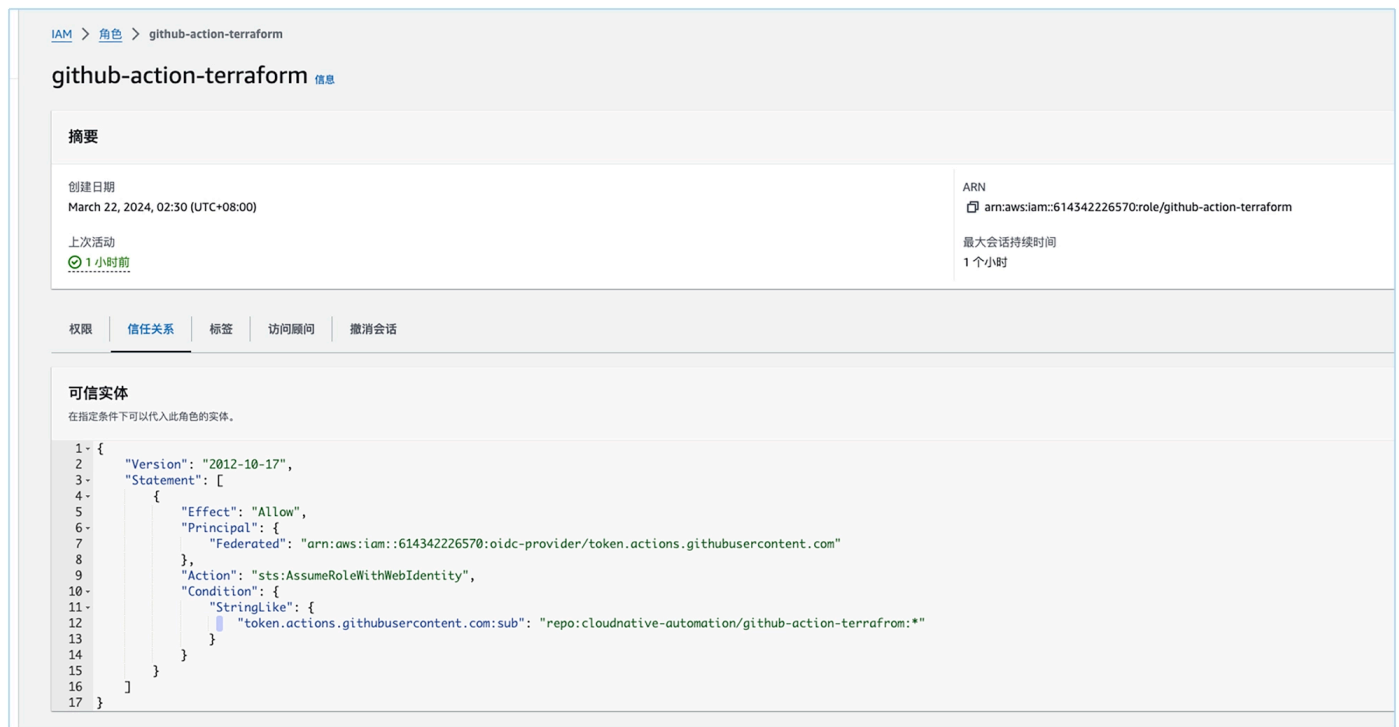
- ☐ 禁用
- ☒ 启用

另外一种创建方法是用 Terraform 代码来生成 S3 的存储桶。这里我的 [GitHub 代码库](#) 中提供了一个创建 S3 存储桶样例代码，你可以自行尝试运行创建。

这里需要注意的是，创建了存储桶以后，我们还无法直接用上它。这是因为 AWS 采用了 ABAC 方式来管理资源，也就是基础属性的访问控制。怎么理解呢？其实 ABAC 这一种授权机制，作用就是根据资源和用户属性来控制用户访问资源的权限。

一般来说，创建好资源以后，还需要为其附加上相应的策略，这样才完成了资源的配置。这些策略一般描述的是，谁可以对这个资源作出哪些操作。对应到我们课程里的例子，就需要给 cloudnative-tfstate-1 这个存储桶做授权。

我们在 IAM 中创建一个角色叫 `github-action-terraform`，这个角色是专门操作 AWS 资源的，具体配置，你可以参考后面的截图。



极客时间

在步骤 1 的信任实体中，填入上一步创建的 oidc-provider 的 arn 号，以及你的 github repo 地址。这里请注意你的环境与我环境有所不同，不要直接复制粘贴，需要根据你的情况调整。

复制代码

```
1 {
```

```
2     "Version": "2012-10-17",
3     "Statement": [
4         {
5             "Effect": "Allow",
6             "Principal": {
7                 "Federated": "arn:aws:iam::{UID}:oidc-provider/token.actions.gith
8             },
9             "Action": "sts:AssumeRoleWithWebIdentity",
10            "Condition": {
11                "StringLike": {
12                    "token.actions.githubusercontent.com:sub": "repo:cloudnative-
13                }
14            }
15        }
16    ]
17 }
```

我们有了操作 AWS 资源的角色，那么我要为这个角色划定权限，也就是赋予一个权限策略，在 AWS IAM 策略中，我们新建一个访问 S3 存储桶的策略，然后填入图上的权限配置，我将权限配置贴在后面，请对照文稿自行查阅。

s3-tfstate 信息

s3-tfstate

策略详细信息

类型

客户托管

创建时间

March 22, 2024, 02:29 (UTC+08:00)

权限

附加的实体

标签

策略版本 (1)

访问顾问

此策略中定义的权限 信息

此策略文档中定义的权限指定了允许或拒绝的操作。要为 IAM 身份(用户、用户组或角色)定义权限，请向其附加一个策略。

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "s3:PutObject",  
8         "s3:GetObject",  
9         "s3:ListBucket"  
10      ],  
11      "Resource": [  
12        "arn:aws:s3:::cloudnative-tfstate-1/*",  
13        "arn:aws:s3:::cloudnative-tfstate-1"  
14      ]  
15    }  
16  ]  
17 }
```



策略的配置如下，也非常好理解，对于 S3 存储桶中的资源

`arn:aws:s3:::cloudnative-tfstate-1` 一般需要有 List、Put 等的权限，这里我在 Action 中加入了 Put 和 List 权限，具体请查看下面的代码。

[复制代码](#)

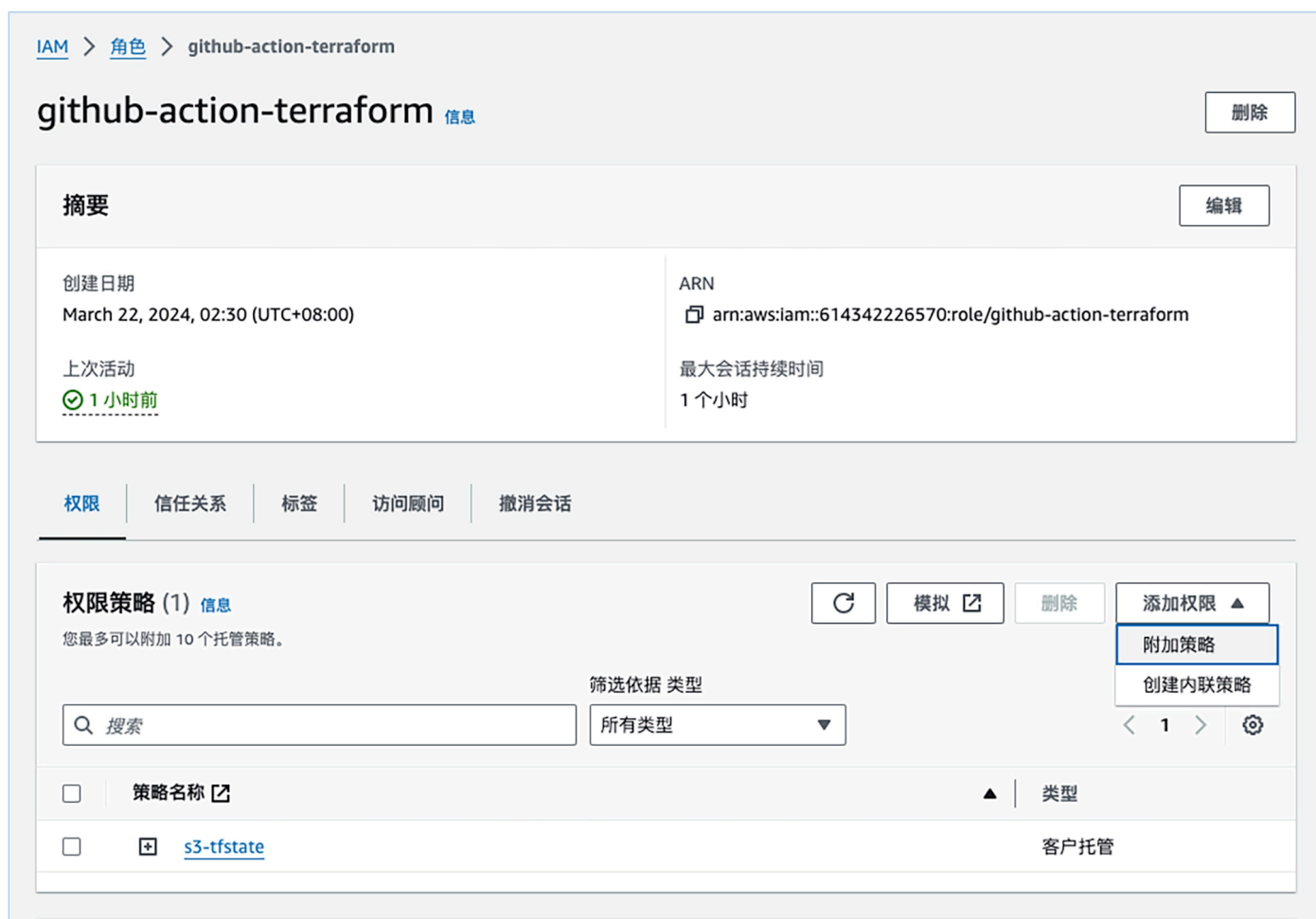
```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",
```

```

6         "Action": [
7             "s3:PutObject",
8             "s3:ListBucket"
9         ],
10        "Resource": [
11            "arn:aws:s3:::cloudnative-tfstate-1/*",
12            "arn:aws:s3:::cloudnative-tfstate-1"
13        ]
14    }
15 ]
16 }

```

最后我们将角色与策略绑定在一起，只要在角色 – 权限策略中，将你刚才新建的策略附加上就可以了。



The screenshot shows the AWS IAM console interface for the role 'github-action-terraform'. The 'Permissions' tab is active, displaying a list of attached policies. A search bar and a filter dropdown (set to 'All types') are visible. A table lists the attached policies, including 's3-tfstate' which is a 'Customer Managed' policy. On the right, the 'Add permissions' dropdown menu is open, showing options to 'Attach policy', 'Create inline policy', and 'Attach managed policy'.

除了自定义策略之外，你也可以复用 AWS 自带的策略，你可以在角色的权限中选择附加策略，这里我附加上 AmazonEC2FullAccess 的策略，在接下来的 EC2 创建中，我们会使用到

这个策略。



极客时间

密钥怎么处理？

现在，我们建好了 S3 存储桶，那么 Github Action 是怎么把状态文件存进去的呢？我们这就来看看 S3 要怎么使用。

因为 Terraform 里已经支持了将状态文件直接存储进 S3，所以我们只要在 terraform init 的时候配置 backend storage 的存储桶以及相应权限即可。命令如下：

复制代码

```
1 terraform init -backend-config="bucket=${AWS_BUCKET_NAME}" -backend-config="key=${AWS_BUCKET_KEY_NAME}"
```

我们需要注意的是，这条命令里有一些变量，比如 **AWS_BUCKET_NAME**，**AWS_REGION** 等。这些变量需要从 GitHub 上通过 Secret 注入到执行过程。

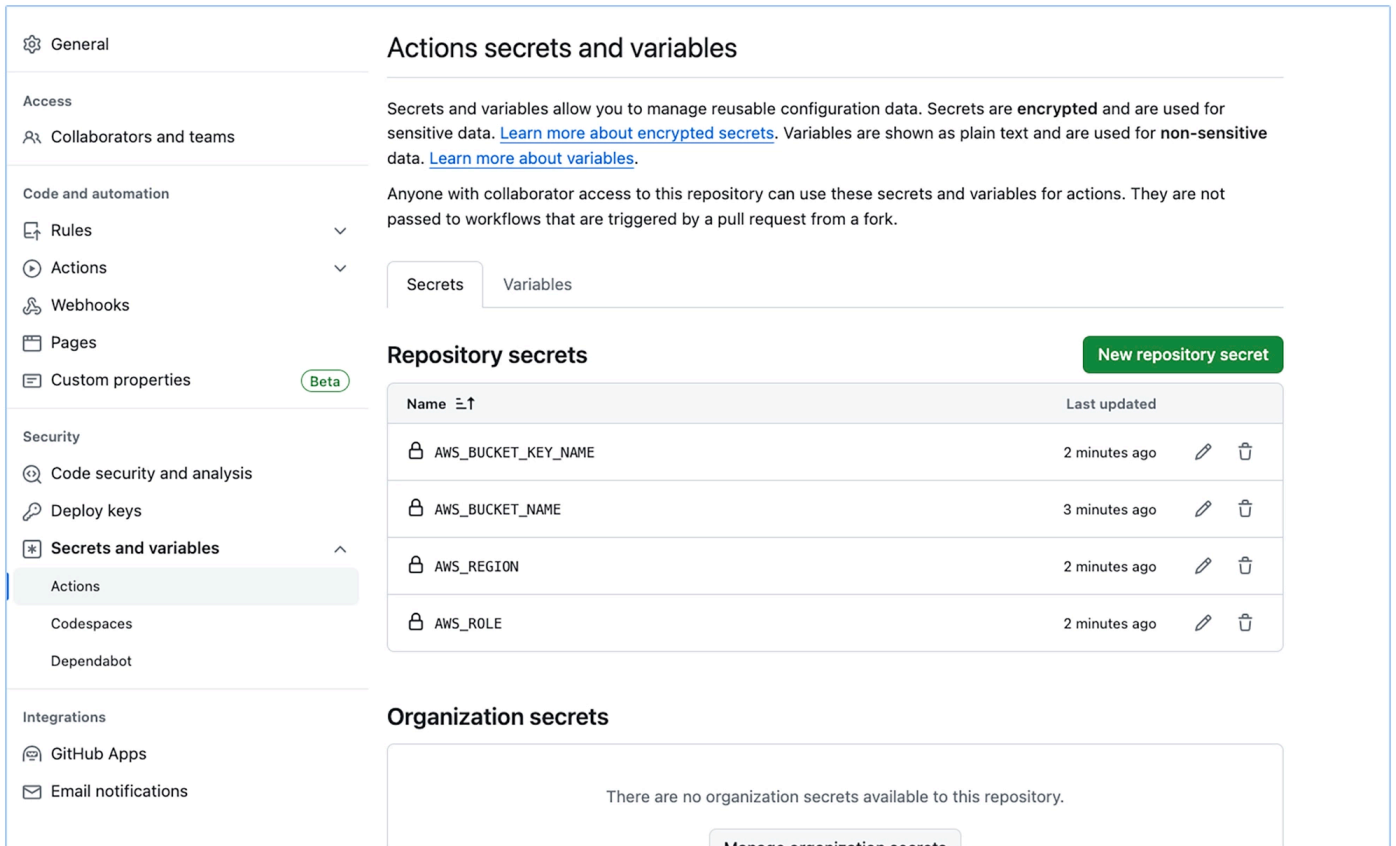
这一步的操作发生在 GitHub 上，过程也很简单。我们只需要点击代码仓库的 setting，找到左下方的 Secret and variables，在 Actions 中添加这几个 Secret。

- AWS_BUCKET_NAME，存储桶名称，这里我们填入上一步中创建的存储桶的名称 cloudnative-tfstate-1
- AWS_BUCKET_KEY_NAME，这里指的是你在存储桶中存储文件的名称，这里我们填入的是 eks.tfstate，稍后你可以在存储桶中看到这个文件。

AWS_REGION：存储桶所在的区域，这里我们填入的是 ap-east-1。

AWS_ROLE：存储桶角色的 ARN，这里可以从创建好的存储桶的属性中找到，这里我们填入 `arn:aws:s3:::cloudnative-tfstate-1`。

在 Github 的界面中，最终看起来像下图一样，就表示我们成功完成了配置。



Github Action 的配置怎么写？

好，现在最关键的一步来了，也就是 GitHub Action 的配置要怎么写？完成这一步，我们才能真正用 GitHub Action 来管理 Terraform 代码。

GitHub Action 中有这么几个重要的字段，on、env 还有 job。on 表示在哪一个 Git 分支上执行。

env 是环境变量，例如我们需要引入上一步中 Region 的这个变量，就可以这样写：

 复制代码

```
1 env:
2   TF_LOG: INFO
3   AWS_REGION: ${ secrets.AWS_REGION }
```

而在 job 这个字段里，就是我们定义真正执行操作步骤的地方。了解了重要字段，我们就可以动手写代码了。

首先，我们需要对环境做一些初始化，例如将代码 checkout 出来，配置 Terraform 的版本，环境等等，我将样例代码贴在了后面。

 复制代码

```
1   steps:
2     - name: Git checkout
3       uses: actions/checkout@v3
4
5     - name: Configure AWS credentials from AWS account
6       uses: aws-actions/configure-aws-credentials@v1
7       with:
8         role-to-assume: ${ secrets.AWS_ROLE }
9         aws-region: ${ secrets.AWS_REGION }
10        role-session-name: GitHub-OIDC-TERRAFORM
11
12    - name: Setup Terraform
13      uses: hashicorp/setup-terraform@v2
14      with:
15        terraform_version: 1.2.5
```

其次，GitHub Pull Request 中有两个主要动作。第一是创建 Pull Request，提交代码并审核；第二是在代码审核完成之后，合并代码。

相应地，我们在第一步创建 Pull Request 的时候，就需要运行 terraform plan 来检查这次提交的代码是否符合预期。然后在第二步合并代码的时候，通过 GitHub Action 运行 terraform apply，真正执行基础设施上的变更。

GitHub Action 的配置很长，为了让你聚焦重点，这里我将两步的核心代码贴在这里，帮助你理解核心用法。GitHub Action 中定义 Terraform plan 这一步，它的触发条件是 `github.event_name == 'pull_request'`。

 复制代码

```
1 - name: Terraform Plan
2   id: plan
3   run: terraform plan -no-color
4   if: github.event_name == 'pull_request'
5   continue-on-error: true
```

接着，我们来看 GitHub Action 中定义 Terraform Apply 这一步。它的触发条件是 `github.ref == 'refs/heads/main' && github.event_name == 'push'`，含义是当 main branch 有 push 的这个动作时候，就执行 terraform apply 这个命令。

 复制代码

```
1 - name: Terraform Apply
2   if: github.ref == 'refs/heads/main' && github.event_name == 'push'
3   run: terraform apply -auto-approve -input=false
```

完整的 GitHub Action 的配置，请参考 [🔗 GitHub 仓库里的代码](#)。

万事俱备，运行管理

这一切都设置完成之后，就到了收割战果的时刻，我们提交一个 PR 来运行一下。

Add more commits by pushing to the **test** branch on [cloudnative-automation/github-action-terrafrom](#).



**Some checks haven't completed yet**[Hide all checks](#)

1 in progress check

 **Terraform action / deploy (pull_request)** *In progress — This check has started...*[Details](#)

**This branch has no conflicts with the base branch**
Merging can be performed automatically.

Merge pull request

You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

 极客时间

从图里我们可以看到，当 Pull Request 提交的时候，自动触发了 Github Actions 的流水线，此时正在运行的是 terrafrom init 以及 terrafrom plan 的步骤。



github-actions bot commented 1 minute ago

Terraform Format and Style  failure

Terraform Initialization  success

Terraform Validation  success

► Validation Output

Terraform Plan  success

▼ Show Plan

terraform

aws_ssm_parameter.foo: Refreshing state... [id=foo]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

aws_instance.example will be created
+ resource "aws_instance" "example" {
+ ami = "ami-038354cc7b631cd6b"
+ arn = (known after apply)
+ associate_public_ip_address = (known after apply)
+ availability_zone = (known after apply)
+ cpu_core_count = (known after apply)
+ cpu_threads_per_core = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ get_password_data = false
+ host_id = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
+ id = (known after apply)
+ instance_initiated_shutdown_behavior = (known after apply)
+ instance_lifecycle = (known after apply)
+ instance_state = (known after apply)
+ instance_type = "t2.micro"
+ ipv6_address_count = (known after apply)
+ ipv6_addresses = (known after apply)
+ key_name = (known after apply)
+ monitoring = (known after apply)
+ outpost_arn = (known after apply)

从这个图中，我们可以看到，当 Pull Request 被 merge 的时候，自动触发了 Github Actions 的流水线，此时正在运行的是 terraform apply 这一步。

```

162     + enable_resource_name_dns_a_record    = (known after apply)
163     + enable_resource_name_dns_aaaa_record = (known after apply)
164     + hostname_type                        = (known after apply)
165   }
166   + root_block_device {
167     + delete_on_termination = (known after apply)
168     + device_name           = (known after apply)
169     + encrypted             = (known after apply)
170     + iops                  = (known after apply)
171     + kms_key_id            = (known after apply)
172     + tags                  = (known after apply)
173     + tags_all              = (known after apply)
174     + throughput            = (known after apply)
175     + volume_id             = (known after apply)
176     + volume_size           = (known after apply)
177     + volume_type           = (known after apply)
178   }
179 }
180 Plan: 1 to add, 0 to change, 0 to destroy.
181 2024-03-31T03:20:37.851Z [INFO] backend/local: apply calling Apply
182 2024-03-31T03:20:37.852Z [INFO] provider: configuring client automatic mTLS
190 2024-03-31T03:20:37.974Z [INFO] provider.terraform-provider-aws_v5.43.0_x5: configuring server automatic mTLS: timestamp=2024-03-31T03:20:37.974Z
191 2024-03-31T03:20:38.769Z [INFO] provider.terraform-provider-aws_v5.43.0_x5: Retrieved credentials: tf_rpc=ConfigureProvider @module=aws.aws
b7a7-7601-2b68-f816374779fc @caller=github.com/hashicorp/aws-sdk-go-base/v2@v2.0.0-beta.50/logging/tf_logger.go:39 tf_mux_provider=*schema.G
timestamp=2024-03-31T03:20:38.767Z
192 2024-03-31T03:20:39.407Z [INFO] provider.terraform-provider-aws_v5.43.0_x5: Retrieved caller identity from STS: tf_mux_provider=*schema.GRP
tf_req_id=fb699266-b7a7-7601-2b68-f816374779fc @caller=github.com/hashicorp/aws-sdk-go-base/v2@v2.0.0-beta.50/logging/tf_logger.go:39 @modul
193 2024-03-31T03:20:39.417Z [WARN] Provider "registry.terraform.io/hashicorp/aws" produced an invalid plan for aws_instance.example, but we are
194 The following problems may be the cause of any confusing errors from downstream operations:
195   - .source_dest_check: planned value cty.True for a non-computed attribute
196   - .user_data_replace_on_change: planned value cty.False for a non-computed attribute
197   - .get_password_data: planned value cty.False for a non-computed attribute
198   - .root_block_device: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
199   - .capacity_reservation_specification: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
200   - .maintenance_options: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
201   - .metadata_options: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
202   - .ephemeral_block_device: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
203   - .network_interface: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
204   - .efs_block_device: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
205   - .private_dns_name_options: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
206   - .cpu_options: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
207   - .enclave_options: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
208   - .instance_market_options: attribute representing nested block must not be unknown itself; set nested attribute values to unknown instead
209 2024-03-31T03:20:39.417Z [INFO] Starting apply for aws_instance.example
210 aws_instance.example: Creating...
211 aws_instance.example: Still creating... [10s elapsed]
212 2024-03-31T03:20:53.655Z [WARN] Provider "provider[\"registry.terraform.io/hashicorp/aws\"]" produced an unexpected new value for aws_insta
plugin SDK.
213 The following problems may be the cause of any confusing errors from downstream operations:
214   - .hibernation: was null, but now cty.False
215   - .credit_specification: block count changed from 0 to 1
216 aws_instance.example: Creation complete after 15s [id=i-065e86b2baddf270e]
217

```

最后，我们在 AWS 的页面上可以看到 EC2 实例创建成功了。

i-065e86b2baddf270e 的实例摘要 信息

less than a minute 前更新

实例 ID

 i-065e86b2baddf270e

IPv6 地址

—

主机名类型

IP 名称: ip-172-31-7-185.ap-east-1.compute.internal

应答私有资源 DNS 名称

—



同时在 S3 存储桶中，我们可以看到 Terraform 的状态文件的不同版本，达到了安全地存储我们的管理文件，又能让我们拥有版本追溯的要求。

Amazon S3 > 存储桶 > cloudnative-tfstate-1 > eks.tfstate

eks.tfstate 信息

属性 权限 版本

版本 (4)

版本 ID	类型	上次修改时间	大小	存储类
 ZP6P8JXFBGL5mP0cz8Yt8lQUVwJvvi (当前版本)	tfstate	2024年3月31日 am11:20:55 CST	4.6KB	标准
 raquybttrQwNLUX_43i4yKtYSFLwRtGw	tfstate	2024年3月31日 am10:45:49 CST	155.0B	标准
 Go74vKxhQL8mYtW1tUSLYNGrgR2RohX	tfstate	2024年3月22日 am3:54:00 CST	995.0B	标准
 LgK9SDewaxtWtGY5v6VPRVtTwz1LFP	tfstate	2024年3月22日 am2:40:49 CST	996.0B	标准



这样，我们就初步完成了用 Github Action 来管理 Terraform 的整个过程。这时，我们真正地将 IaC 完整地实现了。

总结

我们来做个总结。

通过今天的学习，我们应该理解了 GitOps 模式是如何工作的，GitOps 是一种使用 Git 仓库作为 IaC 代码的中央存储库的模式，并使用 DevOps 中的 CI/CD 的形式来管理它。

理解原理只是第一步，今天的重点是配置实操，这个过程掌握了，你才能熟练地使用 GitHub Actions 来管理 Terraform 代码。

我们来回顾一下其中的关键步骤。首先，我们在 AWS 上配置了 Github OIDC，让 GitHub Action 可以通过 OIDC 认证将 AWS 权限安全地转移给 GitHub Action。

然后，我们创建了 S3 存储桶用于保存 terraform 状态文件，并且在创建 S3 存储桶的过程中，我们学习了 AWS IAM 是如何配置的，同时我们在 GitHub 上，利用了 Github Secret 的这个功能，存储了 S3 存储桶的一些相关信息，通过变量的形式传递给 Github Action 流水线。

最后，我们仓库中 `.github/workflows` 目录下创建一个新的 YAML 文件，来完成一个 GitHub Actions 工作流的设置。在这个 YAML 文件中，我们可以定义我们的工作流，包括触发的事件（例如 push 或 pull request）、运行的环境和执行的步骤。

其实整个配置的过程并不简单，尤其是 Github Action 的配置文件的编写，需要不断修改与调试才能达到满意的效果，希望你能够跟着课程实际操作一遍，加深对整个过程的理解。

思考题

课程里，我们将 Terraform 状态文件存放在 S3 这步中，S3 存储桶是我们手动在页面上创建出来的，你能否自己写一段 Terraform 代码，通过 GitHub Action 来创建出 S3 存储桶呢？

欢迎在评论区里给我留言。如果这一讲对你有启发，别忘了分享给身边更多朋友。

AI智能总结

1. GitHub Actions是一个CI/CD工具，特别适合与GitOps结合使用，可以帮助管理和部署IaC代码。
2. GitHub Actions可以通过OIDC的方式与AWS账户进行认证，确保安全性和高效性。
3. 使用GitHub OIDC提供程序并仅允许在特定的GitHub操作中运行凭证，可以更加安全且高效地与AWS进行认证。
4. 在GitHub Action中，需要将Terraform的状态文件存在AWS的S3中，以确保安全存储和版本追溯能力。
5. 创建S3存储桶的方式有两种，可以通过Web Console手动创建，也可以使用Terraform代码生成。
6. 在AWS中，需要为S3存储桶附加相应的策略，以控制用户对资源的访问权限。
7. GitHub Action可以通过Secret注入到执行过程中，以配置Terraform init的backend storage的存储桶和相应权限。
8. 通过GitHub Actions和AWS的认证以及S3存储桶的配置，可以实现管理和部署Terraform代码的自动化操作。
9. GitHub Actions的结合使用，可以提高管理、部署工作的效率，实现IaC与GitOps的无缝衔接。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (4)

最新 精选



陈斯佳

2024-04-24 来自加拿大

老师 关于AWS如何授权Github Action 我还是不太理解 上面那一步在AWS的IAM里配置Github token url的同时 是不是还需要我登入github认证？ 还是说 那个url是统一的 只要你指明github id 那只要是github这个id发来的请求都是合法的？

作者回复: Hi 陈同学

这里是GitHub action去调用AWS API，而AWS的IAM中对repo:cloudnative-automation/github-action-terraform:*做了放行，policy中的规则如下：

```
"token.actions.githubusercontent.com:sub": "repo:cloudnative-automation/github-action-terraform:*"
```



黑石

2024-06-03 来自澳大利亚

```
"token.actions.githubusercontent.com:sub": "repo:cloudnative-automation/github-action-terrafrom:*" } } } }
```

老师，这里的repo要改成自己的repo 吗



Dexter

2024-04-25 来自浙江

开通EKS是不是需要额外的费用？

共 1 条评论 >



橙汁

2024-04-08 来自北京

很不错，解答了我上节课后的疑问，因为我都是在本地apply所以会让手动确认，action后只要pr通过merge到main就生成，也很省事

